AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 9.6: Lecture 10 Preview, Wrap-up, and Appendix

Zhigang Feng

Spring 2026

Topic 9.6 Agenda

1. **Lecture 10 Preview** — case studies on the horizon
2. **Three Econometric Roles** — signal, outcome, instrument
3. **Wrap-up** — six criteria, when not to use an agent, what stays human
4. **Appendix**

Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → **T6 Wrap.**

Lecture 10 Preview

From workflow logic to concrete case studies

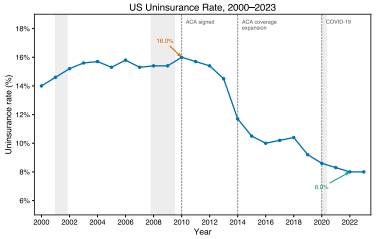
Lecture 10 Preview: Four Concrete Workflow Patterns

Pattern	Lecture 10 case	What students will see
Reusable skill	Paper-review workflow	staged quality control, quote checks, editorial filter
Empirical pipeline	MEPS uninsurance	harmonization, survey weights, benchmark validation, figures
Public-web dataset	Fellows metadata	source restrictions, provenance, missingness discipline
Full-lifecycle collaborator	Glue-work research project	theory, pipeline design, critique, claims discipline

Why this matters: Lec. 9 gives the workflow logic. Lec. 10 shows how the same logic changes shape across different research artifacts.

One Visual Hook from Lecture 10: MEPS Uninsurance

Case Study 2 asks: who is uninsured in the United States, 2000–2023, and how did the ACA change the distribution?



Why keep this figure here: it is an accessible policy example for economists outside macro, and it shows what a validated empirical artifact looks like after the workflow succeeds.

Lec. 10 adds the full pipeline: prompts, schema harmonization, survey weights, external benchmarks, and subgroup figures.

Three Econometric Roles in Agent-Assisted Research

Lec07 T4 introduced a taxonomy: every text-derived variable plays one of three roles — signal, outcome, or instrument. The four Lec10 patterns get sharper when we apply that lens.

Pattern	Signal	Outcome	Instrument
Reusable skill (paper review)	methodological rigor	review quality	staged evaluation rubric
Empirical pipeline (MEPS)	uninsurance rates	health-coverage estimates	survey-weighted harmonization
Public-web dataset (fellows)	field assignment	provenance log	source-restriction rules
Full-lifecycle collaborator (glue-work)	research idea	validated claims	agent + domain expert critique

Same agent toolkit, four research designs. The taxonomy makes you ask: what role does the agent's output play in your inference?

What to Watch for in Lecture 10

- **Paper review:** how a reusable skill turns a vague prompt into a staged quality-control workflow
- **MEPS pipeline:** how an empirical question becomes a validated panel and figure set
- **Fellows dataset:** how provenance and missingness rules matter as much as code
- **Glue-work project:** how agentic AI can act as a full-lifecycle collaborator without replacing human claims discipline

Bridge from Lec. 9 to Lec. 10: today's lecture explains the logic of the workflow; next lecture shows the artifacts it can produce.

Wrap-up

What the workflow changes and what remains human

Six Criteria for Any AI-Assisted Research Output

Before accepting a script, figure, or table the agent produced, run it past six questions.

☐ **1. Reproducibility**	Pinned model, prompt, seed — a co-author can rerun from a clean clone.
☐ **2. Verifiability**	Math, data flow, and magnitudes are independently re-checkable.
☐ **3. Auditability**	Full prompt/output log; a referee can trace where logic diverged from spec.
☐ **4. Cost**	Disclosed dollars (and replicator cost), not just your own.
☐ **5. Speed**	Wall-clock and human-hour saving, net of validation and debugging.
☐ **6. Skill investment**	Does the team build durable capabilities, or quietly atrophy them?

If any answer is "no" or "I don't know," the artifact is not yet research-grade.

Three Things AI Cannot Do — Regardless of Capability

- **Identification.**
  Choosing the variation that answers the causal question.
  An LLM does not know which shock is exogenous, which instrument is valid, or which event window is contaminated. It can describe identification strategies; it cannot *certify* one for your setting.

- **Welfare.**
  Choosing the objective function that defines "good."
  Pareto weights, planner discount rates, equity–efficiency trade-offs — these are normative choices the researcher must own.

- **Judgment.**
  Choosing which question is worth asking.
  AI optimizes against a stated target; only economists decide which target is interesting, novel, and policy-relevant.

These are not gaps that scale away with model size — they are categorically outside the system.

When NOT to Use an Agent

The six criteria can also serve as a boundary: there are research tasks where the agent overhead exceeds the benefit, or where the failure mode is too costly.

- **One-line scripts and isolated edits.** A two-line `sed` or one-shot plot has no validation surface for the harness to add value. Open the editor.
- **Deeply proprietary or undocumented pipelines.** If the conventions live in one person's head, the agent has nothing to read; the time you would spend documenting the pipeline well enough to brief the agent is the same time you would spend just fixing the thing.
- **Strict deterministic guarantees.** For numerical reproducibility audits, regulatory pipelines, or replication packages, model nondeterminism is a hazard the harness cannot fully suppress. Run it pinned; do not let the agent retry.
- **Restricted-data analyses where prompts may leak.** If pasting one column would violate IRB or HIPAA, the agent does not belong in the loop; T5's security workarounds apply.
- **Tasks where you do not yet have a benchmark.** Without a way to validate, faster output just means faster wrong. Define the V0 benchmark first; *then* bring in the agent.

Practical rule: the agent is most useful when the work is bounded, validated, and repeatable. None of those? Skip the agent.

Trust but Verify

1. Reproduce one statistic by hand
2. Cross-check one result with another package or script
3. Test a limiting case or zero-variance benchmark
4. Read the code line by line where the economics enters
5. Ask whether the magnitude and sign make sense

This is not optional ceremony. It is what turns fast code into defensible research.

What Changes for Economic Research

- The distance from question to first result shrinks
- More specifications become cheap enough to try
- Reproducibility can improve because workflow is written down
- The premium shifts away from typing speed and toward design, judgment, and validation

What stays human: asking the question, choosing the design, knowing the data, and interpreting the output.

You Are the Research Architect

Agents can lay bricks quickly.

You still design the structure.

- Weak specification produces wrong models
- Weak algorithm choice produces slow or misleading computation
- Weak numerical judgment produces fragile results
- Weak code literacy produces unverified output

The lecture is not about surrendering expertise. It is about reallocating effort toward the parts of research that remain scarce.

First Exercise and Looking Ahead

Best first exercise: take one old project and ask an agent to help clean it up safely.

- put the project under Git
- create a simple `CLAUDE.md`
- identify one repeated workflow worth encoding
- rerun the cleaned project and compare outputs

Lec. 10: we return to case studies and let students run a full workflow themselves.

If you want setup details for the lab, see the appendix.

Appendix

Setup notes and dated product details

Appendix A: Installing One Terminal Agent (Dated)

Three operational steps, illustrated with Claude Code on macOS:

1. **Install the runtime** (Node.js for Claude Code; Python for Aider; analogous for others):

```
brew install node  # or download from nodejs.org (v18+)
```

2. **Install the agent CLI:**

```
npm install -g @anthropic-ai/claude-code   # Claude Code
# pip install codex-cli                    # OpenAI Codex CLI
# pip install aider-chat                   # Aider
```

3. **Verify:** `claude -version`, then `cd your-project-folder && claude`.

Mac + OneDrive note: always `cd` into the project folder before launching the agent. Running from a `/Volumes/...` path can trigger permission errors.

Dated note: subscription tiers, model IDs, and install commands change every few months. Recheck the official docs before teaching or running the lab.

Appendix A2: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent and confirm it launches
- Create a project folder with a project-specific system prompt (e.g., `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`), `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	`/compact`
Clear session	`/clear`
Interrupt	`Esc`
Slash menu	`/`

Minimal project-system-prompt

```
# CLAUDE.md / AGENTS.md / GEMINI.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```

Appendix C: Dated Comparisons and Resources

As of April 2026, examples of terminal-agent workflows include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

Good places to recheck before teaching:

- official Claude Code documentation
- official OpenAI Codex CLI documentation
- course notes and lab instructions

Teaching rule: keep dated product comparisons in the appendix, not in the conceptual core of the lecture.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.